

tf.RaggedTensorSpec Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/RaggedTensorSpec

Type specification for a tf.RaggedTensor.

Function Signatures

```
tf.RaggedTensorSpec( shape=None, dtype=tf.dtypes.float32,  
    ragged_rank=None, row_splits_dtype=tf.dtypes.int64,  
    flat_values_spec=None )
```

Code Examples

```
tf.RaggedTensorSpec(  
    shape=None,  
    dtype=tf.dtypes.float32,  
    ragged_rank=None,  
    row_splits_dtype=tf.dtypes.int64,  
    flat_values_spec=None  
)
```

```
tf.RaggedTensorSpec(  
    shape=None,  
    dtype=tf.dtypes.float32,  
    ragged_rank=None,  
    row_splits_dtype=tf.dtypes.int64,  
    flat_values_spec=None  
)
```

```
tf.dtypes.float32
```

```
tf.dtypes.int64
```

```
rt = tf.ragged.constant([[ "a" ], [ "b", "c" ]], dtype=tf.string)  
tf.type_spec_from_value(rt).dtype  
tf.string
```

```
rt = tf.ragged.constant([[ "a" ], [ "b", "c" ]], dtype=tf.string)
```

```
tf.type_spec_from_value(rt).dtype
```

Documentation

Type specification for a `tf.RaggedTensor`.

Inherits From: `TypeSpec`, `TraceType`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.RaggedTensorSpec`

Used in the notebooks

- Ragged tensors
 - Subword tokenizers

Attributes

Defaults to `shape.ndims - 1`.

In particular, all values that are compatible with this `TypeSpec` must be an instance of this type.

Methods

`## experimental_as_proto`

[View source](#)

Returns a proto representation of the TypeSpec instance.

Do NOT override for custom non-TF types.

experimental_from_proto

[View source](#)

Returns a TypeSpec instance based on the serialized proto.

Do NOT override for custom non-TF types.

experimental_type_proto

[View source](#)

Returns the type of proto associated with TypeSpec serialization.

Do NOT override for custom non-TF types.

from_value

[View source](#)

is_compatible_with

[View source](#)

Returns true if spec_or_value is compatible with this TypeSpec.

Prefer using "is_subtype_of" and "most_specific_common_supertype" wherever possible.

is_subtype_of

[View source](#)

Returns True if self is a subtype of other.

Implements the `tf.types.experimental.func.TraceType` interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

`most_specific_common_supertype`

[View source](#)

Returns the most specific supertype `TypeSpec` of self and others.

Implements the `tf.types.experimental.func.TraceType` interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

`most_specific_compatible_type`

[View source](#)

Returns the most specific `TypeSpec` compatible with self and other. (deprecated)

Deprecated. Please use `most_specific_common_supertype` instead.
Do not override this function.

`__eq__`

[View source](#)

Return `self==value`.

`__ne__`

[View source](#)

Return `self!=value`.